JP Morgan Financial Analysis:

-By Sushant Shirsekar

Pre-Requisites: Adding data to the Python

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_excel('jp_morgan_dataset.xlsx')
print(df.head())
print(df.info())
```

Description:

1. Imported libraries for data handling and visualization.
2. Loaded Excel dataset into df.
3. Used head() to preview data and info() to check structure and missing values.

Task 1: Data Cleaning and Formatting

Code:

```
# Treated special characters or non-numeric entries from
financial fields.

# Converted currency amounts into numerical format.

df['TransactionAmount'] =
df['TransactionAmount'].replace('[₹$,]', '',
regex=True).astype(float)

df['AccountBalance'] = df['AccountBalance'].replace('[₹$,]',
'', regex=True).astype(float)

df['RiskScore'] = df['RiskScore'].astype(float)

df[‘TransactionAmount’] = df[‘TransactionAmount’].abs()


# Converted whole number numeric format columns (CreditRating,
TenureMonths) to integer.

df['CreditRating'] = df['CreditRating'].astype('int64')

df['TenureMonths'] = df['TenureMonths'].astype('int64')


# Validatted and Formatted (Default Format) Date Columns.

df['TransactionDate'] = pd.to_datetime(df['TransactionDate'],
errors='coerce')


# Converted all string format columns to a single string
format type (capitalized).

df['AccountType'] =
df['AccountType'].str.strip().str.capitalize()

df['TransactionType'] =
df['TransactionType'].str.strip().str.capitalize()

df['Product'] = df['Product'].str.strip().str.capitalize()
```

```
df['Region'] = df['Region'].str.strip().str.capitalize()
df['Manager'] = df['Manager'].str.strip().str.capitalize()
```

Description:

1. Cleaned financial columns by removing currency symbols (₹, $, commas) and converting them into numerical format.
2. Standardized transaction amounts by converting all values to absolute form for consistency.
3. Converted key numerical columns such as credit rating and tenure into integer format.
4. Formatted date columns into proper datetime structure while handling invalid values.
5. Standardized text-based columns by removing extra spaces and ensuring consistent formatting.

Task 2: Descriptive Transactional Analysis

Code:

```
# Monthly Summary using TransactionType instead of amount sign

Total_Credit_M = df[df['TransactionType'].isin(['Deposit',
'Transfer'])]\

    .groupby(df['TransactionDate'].dt.to_period('M'))['Transac
tionAmount'].sum().round(2)

Total_Debit_M = df[df['TransactionType'].isin(['Withdrawal',
'Payment'])]\

    .groupby(df['TransactionDate'].dt.to_period('M'))['Transac
tionAmount'].sum().round(2)

monthly_summary = pd.DataFrame({

    'Total_Credit': Total_Credit_M,

    'Total_Debit': Total_Debit_M.abs(),  # ensure positive
display

    'Net_Transactions': Total_Credit_M - Total_Debit_M.abs()

}).round(2).fillna(0)

monthly_summary.index = monthly_summary.index.to_timestamp()

monthly_summary


# Yearly Summaries of Total Credits, Debits, and Net
Transaction Volume

Total_Debit_Y = df[df['TransactionType'].isin(['Withdrawal',
'Payment'])]\

    .groupby(df['TransactionDate'].dt.to_period('Y'))['Transac
tionAmount'].sum().abs()

Total_Credit_Y = df[df['TransactionType'].isin(['Deposit',
'Transfer'])]\

    .groupby(df['TransactionDate'].dt.to_period('Y'))['Transac
tionAmount'].sum()

yearly_summary = pd.DataFrame({

    'Total_Credit': Total_Credit_Y,
```

```
    'Total_Debit': Total_Debit_Y,
    'Net_Transactions' : Total_Credit_Y - Total_Debit_Y
}).round(2).fillna(0)
yearly_summary.index = yearly_summary.index.to_timestamp()
print(yearly_summary)

# Total Credit vs Debit Line Chart
plt.figure()
plt.plot(monthly_summary.index,
monthly_summary['Total_Credit'], label='Total Credit')
plt.plot(monthly_summary.index,
monthly_summary['Total_Debit'], label='Total Debit')
plt.xlabel('Month')
plt.ylabel('Amount')
plt.title('Total Credits vs. Debits Over Time (Monthly)')
plt.legend()
plt.tight_layout()
plt.show()

# Total Credit and Debit by Account ID
Total_Credit_Acc = df[df['TransactionType'].isin(['Deposit',
'Transfer'])]\
    .groupby(df['AccountID'])['TransactionAmount'].sum()
Total_Debit_Acc = df[df['TransactionType'].isin(['Withdrawal',
'Payment'])]\
    .groupby(df['AccountID'])['TransactionAmount'].sum()
acc_summary = pd.concat([Total_Credit_Acc, Total_Debit_Acc],
axis=1)
acc_summary.columns = ['Total_Credit', 'Total_Debit']
acc_summary = acc_summary.fillna(0)
acc_summary['Net_Transactions'] = acc_summary['Total_Credit']
- acc_summary['Total_Debit']
print(acc_summary.sort_values(by='Net_Transactions'))
```

```
# Created Top 25% Quantiles and Bottom 25% Quantiles to get
top and bottom performing accounts.

top_acc = acc_summary[acc_summary['Net_Transactions'] >=
acc_summary['Net_Transactions'].quantile(0.75)]

bottom_acc = acc_summary[acc_summary['Net_Transactions'] <=
acc_summary['Net_Transactions'].quantile(0.25)]

print(top_acc.sort_values(by='Net_Transactions',
ascending=False))

print(bottom_acc.sort_values(by='Net_Transactions',
ascending=True))


# Identified and flagged accounts as dormant or inactive if
there is a gap of two months or more between consecutive
transactions.

df = df.sort_values(['AccountID', 'TransactionDate'])

df['Gap'] = df.groupby('AccountID')['TransactionDate'].diff()

df['Dormant'] = df['Gap'] >= pd.Timedelta(days=60)

dormant_acc =
df.groupby('AccountID')['Dormant'].max().reset_index()

print(dormant_acc)
```

Description:

1. Calculated monthly and yearly totals of credits, debits, and net transactions to understand overall trends.
2. Plotted a graph to compare how credits and debits change over time.
3. Calculated total transactions for each account and found net balance (credit - debit).
4. Identified top and bottom accounts using quantiles (top 25% and bottom 25%) based on net transactions.
5. Checked inactivity by finding gaps between transactions and marked accounts as dormant if no activity for 60+ days.

Task 3: Customer Profile Building

Code:

```
# Account Segmentation (No. of Transactions):
# Low (1 to 3),
# Medium (4 to 7) and
# High (8 to 10)

# Defining Transaction Volumes
def segment_volume(volume):
    if volume < 4 and volume > 0:
        return 'Low Volume'
    elif volume < 8 and volume >= 4:
        return 'Medium Volume'
    else:
        return 'High Volume'

txn_freq =
df.groupby('AccountID').size().reset_index(name='TransactionVo
lume')
txn_freq['TxnVolumeSegment'] =
txn_freq['TransactionVolume'].apply(segment_volume)
print(txn_freq.sort_values(by='TransactionVolume').tail(20))

# Segmented customers by average balance.
avg_balance =
df.groupby('AccountID')['AccountBalance'].mean().reset_index(n
ame='AvgBalance')
txn_freq = txn_freq.merge(avg_balance, on='AccountID')
print(txn_freq.sort_values(by='AvgBalance'))
```

```
# Account Segmentation (Average Balance):
# Low (< 50,000),
# Medium (>= 50,000 and < 1,00,000) and
# High (>= 1,00,000)

# Defining Balance Segments
def segment_balance(balance):
    if balance >= 100000:
        return 'High Balance'
    elif balance < 100000 and balance >= 50000:
        return 'Medium Balance'
    else :
        return 'Low Balance'

txn_freq['AvgBalanceSegment'] =
txn_freq['AvgBalance'].apply(segment_balance)
print(txn_freq)

# Created Combined Profile Dataset.
# Net inflow
credit = df[df['TransactionType'].isin(['Deposit',
'Transfer'])] \
    .groupby('AccountID')['TransactionAmount'].sum()
debit = df[df['TransactionType'].isin(['Withdrawal',
'Payment'])] \
    .groupby('AccountID')['TransactionAmount'].sum()
net_flow = (credit - debit).reset_index(name='NetFlow')
# Transaction frequency
txn_freq =
df.groupby('AccountID')['TransactionID'].count().reset_index(n
ame='TxnFrequency')
```

```
# Average balance

avg_balance =
df.groupby('AccountID')['AccountBalance'].mean().reset_index(n
ame='AvgBalance')


# Merge all

profile_df = net_flow.merge(txn_freq, on='AccountID') \

                     .merge(avg_balance, on='AccountID')

print(profile_df)


# Defined thresholds using quantiles

# Used 0.75 quantile to identify TOP 25% accounts (high
values)

high_inflow_threshold = profile_df['NetFlow'].quantile(0.75)

high_freq_threshold =
profile_df['TxnFrequency'].quantile(0.75)


# Used 0.25 quantile to identify BOTTOM 25% accounts (low
values)

low_balance_threshold =
profile_df['AvgBalance'].quantile(0.25)


# Used 0.05 quantile to identify extreme low / near-zero
balance accounts (bottom 5%)

near_zero_threshold = profile_df['AvgBalance'].quantile(0.05)


# Assigned customer segments based on quantiles.
profile_df['Segment'] = 'Normal'

# High net inflow accounts
profile_df.loc[
profile_df['NetFlow'] >= high_inflow_threshold,
'Segment'
] = 'High Net Inflow'

# High transaction frequency but low balance
profile_df.loc[
```

```
(profile_df['TxnFrequency'] >= high_freq_threshold) &
(profile_df['AvgBalance'] <= low_balance_threshold),
'Segment'
] = 'High Frequency Low Balance'

# Near zero or negative balance accounts
profile_df.loc[
profile_df['AvgBalance'] <= near_zero_threshold,
'Segment'
] = 'Near Zero / Negative Balance'

profile_df[['AccountID', 'NetFlow', 'TxnFrequency',
'AvgBalance', 'Segment']]
```

Description:

1. Divided accounts into Low, Medium, and High activity based on number of transactions (1–3, 4–7, 8+).
2. Segmented customers based on average balance into low, medium, and high groups using quantile method.
3. Identified high net inflow accounts using top 10% (quantile method).
4. Built customer profiles by combining transaction frequency, balance, and net flow.
5. Created special groups like:
    - High money inflow accounts
    - High activity but low balance accounts
    - Near-zero or negative balance accounts

Task 4: Financial Risk Identification

Code:

```
# Identified large withdrawals

withdrawals = df[df['TransactionType'].isin(['Withdrawal',
'Payment'])]


# Using 0.25 quantile to capture large withdrawals

large_withdrawal_threshold =
withdrawals['TransactionAmount'].quantile(0.75)


large_withdrawals = withdrawals[

    withdrawals['TransactionAmount'] <=
large_withdrawal_threshold

]


# Number of large withdrawals per account

large_withdraw_freq =
large_withdrawals.groupby('AccountID').size().reset_index(name
='LargeWithdrawalCount')


# Identified frequent large withdrawals using top 25%

freq_threshold =
large_withdraw_freq['LargeWithdrawalCount'].quantile(0.75)

frequent_large_withdrawers =
large_withdraw_freq[large_withdraw_freq['LargeWithdrawalCount'
] >= freq_threshold]

print(frequent_large_withdrawers)
```

```
# Identified overdraft accounts (negative balance)

overdraft_accounts = df[df['AccountBalance'] < 0]

overdraft_flag =
overdraft_accounts.groupby('AccountID').size().reset_index(name=='OverdraftCount')


# Combine risk indicators
risk_df = frequent_large_withdrawers.merge(
overdraft_flag,
on='AccountID',
how='outer'
)

risk_df = risk_df.fillna(0)


# Assigned risk labels

risk_df['RiskFlag'] = 'Normal'

risk_df.loc[

    risk_df['LargeWithdrawalCount'] > 0,

    'RiskFlag'

] = 'Frequent Large Withdrawals'

risk_df.loc[

    risk_df['OverdraftCount'] > 0,

    'RiskFlag'

] = 'Overdraft Risk'

risk_df


# Calculated balance volatility using Coefficient of Variation
(CV)

task4_df = df.groupby('AccountID')['AccountBalance'].agg(

    BalanceStd='std',
```

```
    MeanBalance='mean'
).reset_index()

task4_df['CV'] = task4_df['BalanceStd'] /
task4_df['MeanBalance']
task4_df = task4_df.fillna(0)
task4_df

# Detected anomalies using IQR method
Q1 = df['TransactionAmount'].quantile(0.25)
Q3 = df['TransactionAmount'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

anomalies_iqr = df[
    (df['TransactionAmount'] < lower_bound) |
    (df['TransactionAmount'] > upper_bound)
]
anomalies_iqr

# Number of Anomalies per account
anomaly_count =
anomalies_iqr.groupby('AccountID').size().reset_index(name='An
omalyCount')

# Count anomalies per account
anomaly_summary =
anomalies_iqr.groupby('AccountID').size().reset_index(name='An
omaly_Frequency')
```

```
# Created Volatility flag dataset using Quantile method
volatility_flag = task4_df[['AccountID', 'CV']].copy()

# Used 0.75 quantile to identify high volatility accounts (top
25%)
cv_threshold = volatility_flag['CV'].quantile(0.75)

# Flagged high volatility accounts
volatility_flag['HighVolatility'] = volatility_flag['CV'] >=
cv_threshold

print(volatility_flag)

# Calculated large withdrawals using Quantile Method
withdrawals = df[df['TransactionType'].isin(['Withdrawal',
'Payment'])]
threshold = withdrawals['TransactionAmount'].quantile(0.25)
large_withdrawals = withdrawals[
    withdrawals['TransactionAmount'] <= threshold
]

# Number of large withdrawals per account
withdraw_flag =
large_withdrawals.groupby('AccountID').size().reset_index(name
='LargeWithdrawCount')
print(withdraw_flag)

# Prepared CV data
cv_data = task4_df[['AccountID', 'CV']]

# Used previous task's anomalies detecter
```

```
iqr_flags = anomalies_iqr

# Number of anomalies frequency per account
anomaly_summary = (
    iqr_flags.groupby('AccountID')
    .size()
    .reset_index(name='Anomaly_Frequency')
)

# Merge anomaly and volatility data
risk_profile = anomaly_summary.merge(cv_data, on='AccountID',
how='left')

# Identified accounts with negative balances
negative_balance = (
    df[df['AccountBalance'] < 0]
    .groupby('AccountID')
    .size()
    .reset_index(name='NegativeBalanceCount')
)

# Merged negative balance info
risk_profile = risk_profile.merge(negative_balance,
on='AccountID', how='left')
risk_profile['NegativeBalanceCount'] =
risk_profile['NegativeBalanceCount'].fillna(0)

# Used0 0.75 quantile to identify high volatility accounts
cv_threshold = risk_profile['CV'].quantile(0.75)

# Final risk flag based on multiple conditions
risk_profile['RiskFlag'] = (
```

```
    (risk_profile['Anomaly_Frequency'] >= 2) |
    (risk_profile['CV'] >= cv_threshold) |
    (risk_profile['NegativeBalanceCount'] > 0)
)

# Filter suspicious accounts
suspicious_accounts = risk_profile[risk_profile['RiskFlag'] ==
True]
print(suspicious_accounts)
```

Description:

1. Identified large withdrawals using bottom 25% values.
2. Found accounts with frequent large withdrawals using top 25%.
3. Detected overdraft accounts with negative balances.
4. Measured balance changes using volatility (CV).
5. Used IQR method to detect unusual transactions.
6. Combined all factors to identify suspicious accounts.

Task 5: Exploratory Data Analysis

Code:

```
# 1. Monthly Credit vs Debit Trend
# Line chart to analyze how total credits and debits change
over time
monthly_summary[['Total_Credit',
'Total_Debit']].plot(figsize=(10,5))
# Title and labels
plt.title('Monthly Credit vs Debit Trend')
plt.xlabel('Date')
plt.ylabel('Amount')
# Display the chart
plt.show()


# 3. Customer Distribution by Activity Level
# Group accounts by number of transactions
txn_freq =
df.groupby('AccountID').size().reset_index(name='TransactionVo
lume')
# Assign activity segments (Low, Medium, High)
txn_freq['TxnVolumeSegment'] =
txn_freq['TransactionVolume'].apply(segment_volume)
# Count number of accounts in each segment
segment_counts = txn_freq['TxnVolumeSegment'].value_counts()
# Bar chart for distribution
plt.figure()
plt.bar(segment_counts.index, segment_counts.values)
# Labels and title
plt.xlabel('Transaction Volume Segment')
plt.ylabel('Number of Accounts')
plt.title('Customer Distribution by Activity Level')
plt.tight_layout()
plt.show()


# 4. Transaction Frequency vs Average Balance (Behavior
Analysis)
# Scatter plot to analyze relationship between frequency and
balance
plt.figure(figsize=(6,4))
sns.scatterplot(x=profile_df['TxnFrequency'],
```

```
y=profile_df['AvgBalance'])
# Labels and title
plt.title('Transaction Frequency vs Average Balance')
plt.xlabel('Transaction Frequency')
plt.ylabel('Average Balance')
# Grid for better readability
plt.grid()
# Display plot
plt.show()


# 5. Customer Segmentation Distribution
# Count number of customers in each segment
profile_df['Segment'].value_counts().plot(kind='bar')
# Labels and title
plt.title('Customer Segmentation Distribution')
plt.xlabel('Segment')
plt.ylabel('Count')
# Display chart
plt.show()



# 6. Box Plot (IQR Method for Anomaly Detection)
# Plot to identify outliers in transaction amounts
plt.figure(figsize=(6,6))
sns.boxplot(x=df['TransactionAmount'])
# Labels and title
plt.title('Box Plot of Transaction Amount (IQR Method)')
plt.xlabel('Transaction Amount')
# Display plot
plt.show()


# 7. Distribution of Balance Volatility (CV)
# Histogram with KDE to analyze volatility distribution
plt.figure(figsize=(6,4))

sns.histplot(task4_df['CV'], bins=30, kde=True)

# Labels and title
plt.title('Distribution of Balance Volatility (CV)')
plt.xlabel('Coefficient of Variation')
plt.ylabel('Frequency')
```

Description:

1. Analyzed monthly credit and debit trends using a line chart to understand how money inflow and outflow change over time and identify overall transaction patterns.
2. Compared top and bottom performing accounts using a bar chart to highlight customers with highest and lowest net transactions, helping identify high-value and low-performing accounts.
3. Grouped customers based on transaction activity levels (Low, Medium, High) using a bar chart to understand how active different accounts are.
4. Examined relationship between transaction frequency and average balance using a scatter plot to identify customer behavior patterns such as high activity with low balance or high-value customers.
5. Visualized overall customer segmentation using a bar chart to show distribution of customers across different segments like High Net Inflow, High Frequency Low Balance, etc.
6. Detected unusual or extreme transactions using a box plot (IQR method) to identify outliers that may indicate suspicious financial activity.
7. Analyzed balance volatility using CV distribution with a histogram and KDE curve to identify accounts with unstable balances, which may represent financial risk.

Task 6: Hypothesis Testing

Code:

```
# Step 1: Divide accounts into High and Low transaction groups
using median

median_freq = profile_df['TxnFrequency'].median()

profile_df['TxnSegment'] = profile_df['TxnFrequency'].apply(

    lambda x: 'High' if x > median_freq else 'Low'

)

# Display updated dataframe

profile_df



# Step 2: Extract average balance for both groups

high_vol = profile_df[profile_df['TxnSegment'] ==
'High']['AvgBalance']

low_vol = profile_df[profile_df['TxnSegment'] ==
'Low']['AvgBalance']



# Step 3: Define hypotheses

# H0 (Null Hypothesis): High-volume accounts do NOT have
higher average balance

# H1 (Alternative Hypothesis): High-volume accounts have
higher average balance

# Compare mean values

print("Mean High Volume:", high_vol.mean())

print("Mean Low Volume:", low_vol.mean())
```

```
# Step 4: Perform T-test
from scipy.stats import ttest_ind
t_stat, p_value = ttest_ind(high_vol, low_vol)
# Print p-value
print("P-value:", p_value)


# Step 5: Decision making based on significance level
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")
```

Description:

1. Divided accounts into High and Low transaction groups using the median of transaction frequency to compare different activity levels.
2. Extracted average balances for both groups to analyze whether transaction activity impacts account balance.
3. Defined hypotheses, where the null hypothesis assumes no difference, and the alternative hypothesis assumes that high-volume accounts have higher balances.
4. Calculated mean values of both groups to get an initial understanding of differences in average balance.
5. Performed an independent T-test to statistically compare the two groups and determine whether the difference is significant.
6. Used p-value and significance level (0.05) to make a final decision on whether to reject or accept the null hypothesis.

Video Link: https://drive.google.com/file/d/1jHcjCmoVLMGpVt9RzGAjiSl-TekcXrag/view?usp=sharing